Listing 7.5 Integrating the summarization action based on the input

```
const summarizeText = async (text) => {
  return summarizeDocs([text]);
};
export async function continueConversation(input) {
  const aiState = getMutableAIState();

  try {
    let summary;
    if (input instanceof FormData) {
      const file = input.get('file');
      summary = await processFile(file, file.type);
    } else {
      summary = await summarizeText(input);
    }
    ...
```

The summarizeText function is responsible for handling text summarization.

Checks if the input is an instance of FormData. This indicates that the user has uploaded a file for summarization.

If the input is a file, we extract the file from the FormData object and call the processFile function, passing the file and its MIME type.

The `continueConversation` function orchestrates the overall process by checking if the input is a file or text and calling either the `summarizeText` or the `processFile`.

Both functions call the `summarizeDocs` function that is a wrapper of the code in listing 7.2, with some small adjustments to include the `normalizeDocuments` function introduced in listing 7.3. The output of the `summarizeDocs` function is a string that contains the summary of the list of documents.

Regarding the changes to the UI, it's important to note the modifications made to the `handleSubmit` function within page.js. When a user uploads a document, we encapsulate the selected file in a `FormData` object. This approach ensures that the `continueConversation` function can accurately check the type of input being processed. The relevant code snippet initializes a new `FormData` instance and appends the uploaded file and then calls the `continueConversation`.

Listing 7.6 Uploading a document file to summarization

```
    ...
    const formData = new FormData();
    formData.append('file', selectedFile);

    setConversation((currentConversation) => [
      ...currentConversation,
      { id: generateId(), role: 'user',
display: `Uploaded file: ${selectedFile.name}` },
    ]);

    response = await continueConversation(formData);
```

Creates a new FormData object to hold form data, including file uploads

Adds a new message entry indicating the uploaded file

Calls the continueConversation function with the FormData object to process the uploaded file and continue the conversation

Now that we have covered a basic web application capable of summarizing documents, we can focus on enhancing its reliability and performance.